

AcquirerX Deep-Dive 2/10 — API Gateway

Service: `api-gateway` **Port:** `9090` **Database:** None **Role:** Single entry point — routing, JWT validation, rate limiting, CORS

1. What It Is

The API Gateway is the **single front door** for all client traffic. The frontend, Postman, and any external caller hit *only* `http://localhost:9090/api/v1/...` — never the individual services directly.

Behind the gateway, traffic is fanned out to one of 8 backend services based on URL path. The gateway also enforces cross-cutting concerns that would otherwise have to be duplicated in every service:

- JWT validation (one place, not eight)
 - Rate limiting (per-tier, per-path, per-client)
 - CORS (single policy)
 - Routing (path → service)
 - Token forwarding (gateway forwards user identity headers to downstream)
-

2. Why It Exists

2.1 Without a gateway

Every service exposes its own port. The frontend would need:

```
http://localhost:9099/api/v1/auth/login
http://localhost:9091/api/v1/merchants
http://localhost:9092/api/v1/terminals
http://localhost:9093/api/v1/transactions
... etc
```

Problems:

- Frontend has to know all service URLs (8 different ports)
- CORS config duplicated in 8 services

- JWT validation duplicated in 8 services
- Rate limiting duplicated in 8 services
- Each service exposed to network = larger attack surface
- Refactoring (e.g., merging two services) breaks the frontend

2.2 With a gateway

Frontend hits ONE URL: `http://localhost:9090/api/v1/...`

The gateway is the only public-facing component. Backend services trust headers forwarded by the gateway and don't validate JWT themselves.

3. Tech Stack

- **Spring Cloud Gateway** (reactive, built on Netty + Project Reactor — not servlet-based)
 - **Bucket4j + Caffeine** — token-bucket rate limiting with in-memory cache
 - **JJWT 0.11.x** — for JWT parsing (must match auth-service's version)
 - **Spring Boot Actuator** — health, gateway routes inspection
-

4. The Code Structure

```
api-gateway/
├─ pom.xml
├─ src/main/java/com/acquirerx/gateway/
│   ├── GatewayApplication.java
│   ├── controller/
│   │   └─ CorsCheckController.java
│   ├── ratelimit/
│   │   ├── BucketService.java           (Bucket4j wrapper)
│   │   ├── ClientIdentifier.java        (extracts identity for rate limiting)
│   │   ├── RateLimitFilter.java         (gateway filter)
│   │   ├── RateLimitProperties.java     (binds rate-limit.* config)
│   │   └─ TierResolver.java            (path → tier mapping)
│   └─ security/
│       ├── JwtAuthenticationFilter.java
│       └─ JwtUtil.java
```

5. Routing — How Requests Flow

5.1 Configuration (application.properties)

Routes are declared in order. Gateway evaluates them top-to-bottom and uses the first match.

```
properties

spring.cloud.gateway.routes[0].id=auth-service
spring.cloud.gateway.routes[0].uri=lb://AUTH-SERVICE
spring.cloud.gateway.routes[0].predicates[0]=Path=/api/v1/auth/**

spring.cloud.gateway.routes[1].id=store-terminals
spring.cloud.gateway.routes[1].uri=lb://TERMINAL-SERVICE
spring.cloud.gateway.routes[1].predicates[0]=Path=/api/v1/stores/*/terminals,/api/v1

spring.cloud.gateway.routes[2].id=merchant-service
spring.cloud.gateway.routes[2].uri=lb://MERCHANT-SERVICE
spring.cloud.gateway.routes[2].predicates[0]=Path=/api/v1/merchants/**,/api/v1/stores
```

The `lb://` prefix: This means "load-balanced via Eureka." The gateway resolves `MERCHANT-SERVICE` from Eureka at request time, picks an instance, and forwards the request.

Why route ordering matters: Route `[1]` matches `/api/v1/stores/*/terminals` — terminal sub-routes under a store. This must come BEFORE route `[2]` which matches the broader `/api/v1/stores/**`. Otherwise, `/api/v1/stores/5/terminals` would hit merchant-service (which doesn't have terminal endpoints) and return 404.

5.2 Full route table

Order	ID	Pattern	Target
0	auth-service	<code>/api/v1/auth/**</code>	AUTH-SERVICE
1	store-terminals	<code>/api/v1/stores/*/terminals[/]**</code>	TERMINAL-SERVICE
2	merchant-service	<code>/api/v1/merchants/**</code> , <code>/api/v1/stores/**</code>	MERCHANT-SERVICE
3	terminal-service	<code>/api/v1/terminals/**</code>	TERMINAL-SERVICE

Order	ID	Pattern	Target
4	transaction-service	<code>/api/v1/transactions/**</code> , <code>/api/v1/txns/**</code> , <code>/api/v1/fee-rules/**</code>	TRANSACTION-SERVICE
5	risk-service	<code>/api/v1/risk/**</code>	RISK-SERVICE
6	settlement-service	<code>/api/v1/settlement/**</code> , <code>/api/v1/payout/**</code>	SETTLEMENT-SERVICE
7	ops-service	<code>/api/v1/disputes/**</code> , <code>/api/v1/recon/**</code> , <code>/api/v1/reports/**</code> , <code>/api/v1/notifications/**</code>	OPS-SERVICE
8	iam-service	<code>/api/v1/iam/**</code>	AUTH-SERVICE

6. JWT Validation Filter

6.1 Why it's at the gateway, not each service

Earlier in development, every service had its own JWT filter. This was problematic because:

- 8 places to update if signing logic changes
- Risk of one service drifting (e.g., wrong secret, different expiration logic)
- If someone bypasses the gateway and hits a service directly, they could exploit any inconsistency

Phase E of the migration moved JWT validation to the gateway. Downstream services now just trust the headers the gateway forwards (`X-User-Id`, `X-User-Role`, `X-Username`).

6.2 `JwtAuthenticationFilter.java` — what it does

For every request:

1. Check if the request path is in `gateway.jwt.open-paths` — if yes, skip JWT entirely
2. Extract the `Authorization` header
3. If missing or doesn't start with `Bearer` → return `401 Unauthorized`
4. Strip the `Bearer` prefix to get the token
5. Validate the token using `JwtUtil`:
 - Signature matches the configured secret

- Token isn't expired
- Parses correctly

6. If invalid → return `401`

7. If valid:

- Extract username and role from claims
- Add headers to the forwarded request: `X-Username`, `X-User-Role`
- Forward downstream

6.3 `JwtUtil.java` — gateway version

Almost identical to auth-service's `JwtUtil`:

```
java

private SecretKey getKey() {
    return Keys.hmacShaKeyFor(secret.getBytes());
}

public boolean isValid(String token) {
    try {
        getClaims(token);
        return true;
    } catch (JwtException | IllegalArgumentException e) {
        return false;
    }
}

public String extractUsername(String token) {
    return getClaims(token).getSubject();
}

public String extractRole(String token) {
    return getClaims(token).get("role", String.class);
}
```

Critical: the `jwt.secret` MUST match auth-service's secret exactly. If they differ by even one character, every request will fail with 401.

6.4 Open paths (skip JWT)

```
/api/v1/auth/login
```

```
/api/v1/auth/register  
/api/v1/auth/validate  
/actuator  
/eureka  
/v3/api-docs  
/swagger-ui  
/api/v1/cors-check  
OPTIONS ← all CORS preflight requests
```

`OPTIONS` is in the list because browsers send a CORS preflight `OPTIONS` request before any cross-origin POST/PUT — this preflight has no Authorization header by design and would fail JWT validation otherwise.

7. Rate Limiting (Bucket4j)

7.1 The token bucket algorithm

Each client+path combination gets its own "bucket" of tokens:

- Bucket starts full (e.g., 5 tokens for `strict` tier)
- Every request consumes one token
- Tokens refill at the configured rate (e.g., 5 per minute)
- If bucket is empty → request is rejected with `429 Too Many Requests`

7.2 Tiers (from config)

properties

```
rate-limit.tiers.strict.capacity=5
rate-limit.tiers.strict.refill-tokens=5
rate-limit.tiers.strict.refill-period-seconds=60

rate-limit.tiers.standard.capacity=60
rate-limit.tiers.standard.refill-tokens=60
rate-limit.tiers.standard.refill-period-seconds=60

rate-limit.tiers.generous.capacity=300
rate-limit.tiers.generous.refill-tokens=300
rate-limit.tiers.generous.refill-period-seconds=60

rate-limit.tiers.lenient.capacity=120
rate-limit.tiers.lenient.refill-tokens=120
rate-limit.tiers.lenient.refill-period-seconds=60
```

7.3 Why these tiers exist

Tier	Use case	Why
strict (5/min)	Login, Register	Brute-force protection — 5 wrong passwords/min stops credential stuffing
standard (60/min)	Most CRUD endpoints	Reasonable per-user activity limit
generous (300/min)	Switch authorize, txn-from-auth	A POS terminal might burst many transactions in a minute
lenient (120/min)	Actuator, Swagger	Health checks and developer tooling shouldn't hit limits easily

7.4 The 5 rate-limit classes

`RateLimitProperties.java`

Maps the `rate-limit.*` properties from `application.properties` into a Java config object. Spring's `@ConfigurationProperties` auto-binds. Holds the tier definitions and path-to-tier mappings.

`TierResolver.java`

Given a request path, returns which tier applies. Walks through `rate-limit.paths[i]` patterns in order and returns the first match. Falls back to `standard` if no pattern matches.

```
java
// Pseudo-flow
public String resolveTier(String path) {
    for (PathConfig p : properties.getPaths()) {
        if (pathMatches(p.getPattern(), path)) return p.getTier();
    }
    return "standard";
}
```

Pattern matching uses Spring's `AntPathMatcher` (so `/actuator/**` matches `/actuator/health`).

`ClientIdentifier.java`

Extracts a unique client identity for the bucket key. Probably uses something like:

- IP address from `X-Forwarded-For` header (or remote address fallback)
- Optionally combined with username if authenticated

This means each client gets their own bucket per path — one user spamming login doesn't affect another user.

`BucketService.java`

The `Bucket4j` wrapper. Holds an in-memory map (Caffeine cache) of bucket keys to actual `Bucket` instances. Creates buckets on demand using the appropriate tier configuration.

```
java
// Pseudo-flow
public Bucket getBucket(String clientId, String tier) {
    String key = clientId + ":" + tier;
    return cache.get(key, k -> createBucketForTier(tier));
}
```

The Caffeine cache evicts buckets after some idle period to prevent memory leaks (e.g., if 10,000 unique IPs hit the gateway, you don't want 10,000 buckets sitting in memory forever).

`RateLimitFilter.java`

The actual gateway filter. For every request:

1. Get path → `tierResolver.resolveTier(path)` → e.g., "strict"
2. Get client identity → `clientIdentifier.identify(request)` → e.g., "192.168.1.5"
3. Get bucket → `bucketService.getBucket(clientId, tier)`
4. Try to consume 1 token: `bucket.tryConsume(1)`
5. If success: add headers and forward
 - `X-RateLimit-Limit: 5`
 - `X-RateLimit-Remaining: 4`
 - `X-RateLimit-Tier: strict`
6. If failure: return `429 Too Many Requests` with:
 - `Retry-After: 60` (seconds until tokens refill)

7.5 Path → tier mappings

properties

```
rate-limit.paths[0].pattern=/api/v1/auth/login           → strict
rate-limit.paths[1].pattern=/api/v1/auth/register        → strict
rate-limit.paths[2].pattern=/api/v1/switch/authorize     → generous
rate-limit.paths[3].pattern=/api/v1/txns/from-auth/**    → generous
rate-limit.paths[4].pattern=/actuator/**                 → lenient
rate-limit.paths[5].pattern=/swagger-ui/**              → lenient
rate-limit.paths[6].pattern=/v3/api-docs/**             → lenient
```

Everything not listed → falls through to `standard`.

8. CORS Configuration

8.1 Why CORS matters

Browser fetches from `http://localhost:5173` (frontend) to `http://localhost:9090` (gateway) are cross-origin. Without correct CORS headers, the browser blocks the response. The gateway has to:

1. Tell the browser which origins, methods, and headers are allowed
2. Handle the preflight `OPTIONS` request before each non-simple request
3. Echo back the right `Access-Control-*` headers

8.2 Config

properties

```
spring.cloud.gateway.globalcors.cors-configurations.[/**].allowed-origins=http://localhost
spring.cloud.gateway.globalcors.cors-configurations.[/**].allowed-methods=GET,POST,PUT,DELETE
spring.cloud.gateway.globalcors.cors-configurations.[/**].allowed-headers=*
spring.cloud.gateway.globalcors.cors-configurations.[/**].exposed-headers=Idempotency-Key, Content-Type, Content-Length
spring.cloud.gateway.globalcors.cors-configurations.[/**].allow-credentials=true
spring.cloud.gateway.globalcors.cors-configurations.[/**].max-age=3600
```

8.3 Why each setting

Setting	Reason
<code>allowed-origins</code>	Only specific dev origins. Production would have <code>https://acquirerx.com</code>
<code>allowed-methods</code>	All HTTP verbs the API uses
<code>allowed-headers=*</code>	Allow any request header (Authorization, Idempotency-Key, Content-Type, etc.)
<code>exposed-headers</code>	Headers the <i>frontend JS</i> can read from the response. Without listing, the browser hides them. Includes rate-limit info, idempotency replay flag, trace ID for debugging
<code>allow-credentials=true</code>	Allows cookies/auth headers in cross-origin requests
<code>max-age=3600</code>	Browser caches preflight result for 1 hour, reducing OPTIONS spam

8.4 `CorsCheckController.java`

A debug endpoint at `/api/v1/cors-check` (in open-paths). Hitting it from the browser confirms CORS is working — if you get a CORS error here, your origin isn't whitelisted.

9. Forwarded Headers

When the gateway successfully validates a JWT, it adds these headers to the forwarded request

before sending to downstream services:

Header	Source	Use case
X-Username	JWT <code>sub</code> claim	Downstream services know who's calling
X-User-Role	JWT <code>role</code> claim	Role-based authorization in services

Downstream services read these headers and trust them (they trust that only the gateway can set them, because services are on a private network).

10. Actuator

properties

```
management.endpoints.web.exposure.include=health,info,gateway
management.endpoint.gateway.enabled=true
management.endpoint.health.probes.enabled=true
management.endpoint.health.group.liveness.include=livenessState,ping
management.endpoint.health.group.readiness.include=readinessState,diskSpace
management.endpoint.health.show-details=always
```

Useful endpoints:

Endpoint	What it shows
GET /actuator/health	Basic UP/DOWN
GET /actuator/health/liveness	Is the JVM alive? (for Kubernetes liveness probe)
GET /actuator/health/readiness	Is it ready to serve traffic? (Kubernetes readiness probe)
GET /actuator/gateway/routes	List all configured routes — extremely useful for debugging
GET /actuator/info	Build/git info if configured

11. Filter Execution Order

Spring Cloud Gateway filters run in a specific order:

Request comes in

↓

Pre-filters (in order):

1. CORS handling (if OPTIONS preflight, respond immediately)
2. RateLimitFilter – check tokens, reject if 429
3. JwtAuthenticationFilter – validate JWT, add X-Username/X-User-Role

↓

Route matching → forward to downstream

↓

Response flows back

↓

Post-filters (in reverse order):

- Add CORS headers to response
- Add X-RateLimit-* headers to response

↓

Response goes to client

12. Logging

properties

```
logging.level.org.springframework.cloud.gateway=DEBUG
```

This logs every routing decision — extremely noisy but invaluable when debugging "why is my request going to the wrong service."

13. Failure Scenarios

Scenario	Result
Downstream service is DOWN	Gateway gets connection refused → returns <code>503 Service Unavailable</code>
Downstream times out	Gateway hits its default timeout → returns <code>504 Gateway Timeout</code>
JWT signature invalid	<code>401 Unauthorized</code> (stops at gateway, never hits downstream)
Rate limit exceeded	<code>429 Too Many Requests</code> with <code>Retry-After</code> header

Scenario	Result
Path doesn't match any route	404 Not Found
Eureka is unreachable	Gateway can't resolve lb://SERVICE-NAME → 503

14. Why Bucket4j (vs Other Rate Limiters)

Option	Why not chosen
Spring Cloud Gateway built-in (Redis-based)	Requires running Redis — extra infra
Resilience4J RateLimiter	More for <i>internal</i> call rate limiting, not gateway-style per-client buckets
Bucket4j + Caffeine	In-memory, no extra dependencies, simple, fast, perfect for single-instance gateway in dev/learning

In production with multiple gateway instances, you'd switch to Bucket4j with a distributed backend (Redis/Hazelcast) so all gateway nodes share the same buckets. For now, in-memory is fine.

15. Summary

Aspect	Detail
Purpose	Single entry, JWT, rate limit, CORS, routing
Port	9090
Tech	Spring Cloud Gateway (reactive, Netty-based)
Filters	Rate limit → JWT → Routing
Auth	JWT validation centralized here (Phase E)
Rate limiting	Bucket4j + Caffeine, 4 tiers, per-client buckets
CORS	4 dev origins, 7 exposed headers, credentials allowed

Aspect	Detail
Service discovery	<code>lb://SERVICE-NAME</code> via Eureka
Custom code	<code>JwtAuthenticationFilter</code> , <code>JwtUtil</code> , 5 rate-limit classes, <code>CorsCheckController</code>

Next: Auth Service — IAM, JWT generation, audit logs.